

Primera entrega: Twitch analytics

Grupo Heropass



API	2
Token	3
Scripts	5
streamer.php	6
streams.php	6
topStreams.php	7
Código streamer.php	8
Código streams.php	9
Código topStreams.php	10
Hosting	12
Códigos de respuesta y peticiones	13
Códigos 200	13
Caso 1	13
Caso 2	13
Caso 3	14
Código 400	14
Código 401	14
Código 500	15
Problemas encontrados y soluciones	17

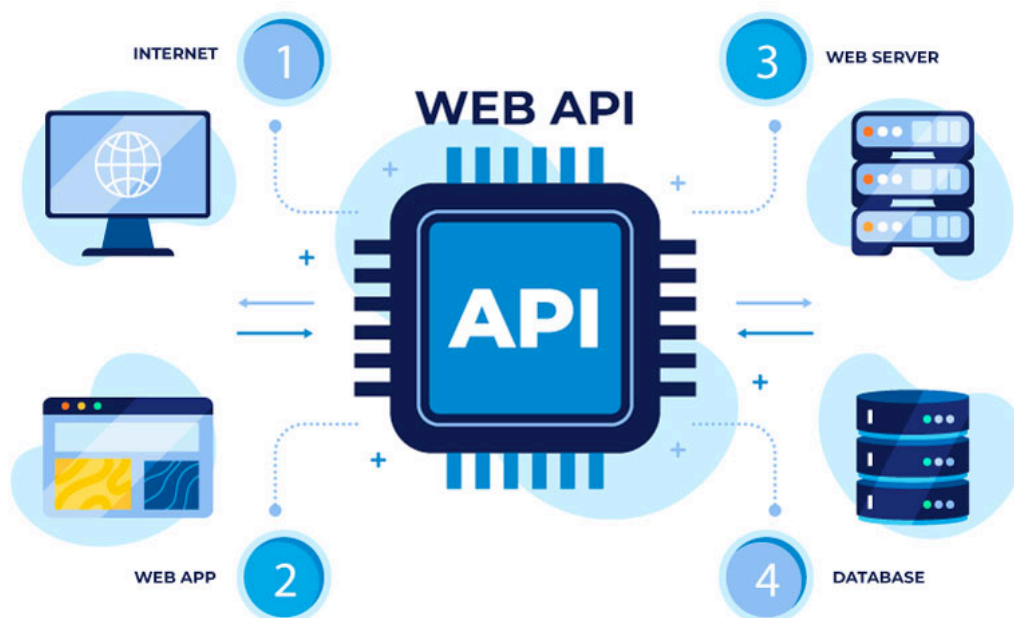
API

Antes de adentrarnos en nuestro proyecto, es esencial resolver uno de los primeros problemas que surgieron al comenzar. Para entender y desarrollar este proyecto, debemos hacernos la pregunta clave: ¿Qué es una API?

Una API, o Interfaz de Programación de Aplicaciones (Application Programming Interface en inglés), es un conjunto de reglas y protocolos que permite la comunicación entre diferentes aplicaciones. Actúa como un intermediario que facilita la interacción y el intercambio seguro y eficiente de datos entre dos programas informáticos.

Para visualizarlo mejor, imagina que una API es como un camarero en un restaurante: tú, como cliente, le das tu pedido al camarero, y él se comunica con la cocina para que preparen tu comida. Una vez que la comida está lista, el camarero te la trae de vuelta. En este caso, el camarero representa la API, que facilita la comunicación entre tú (la aplicación que solicita un servicio) y la cocina (la aplicación que proporciona el servicio).

Ahora que entendemos qué es una API, vamos a explorar cómo se realizan las solicitudes a la API y qué tipo de respuestas proporciona. Sin embargo, antes de abordar esto, necesitamos crear el token necesario para trabajar con la API de Twitch.



Token

Un token es una representación digital de un activo o utilidad específica. En el contexto de las APIs, un token de acceso es una cadena de caracteres que permite a una aplicación autenticarse y acceder a recursos protegidos en nombre de un usuario o de la propia aplicación. Los tokens son fundamentales para la seguridad y la gestión de permisos en las interacciones entre aplicaciones.

Para obtener un token de acceso de Twitch, primero registramos nuestra aplicación en el [portal de desarrolladores](#) de Twitch para obtener un `client_id` y un `client_secret`.

Luego, utilizamos estas credenciales para solicitar un token de acceso mediante flujos de autenticación, como el flujo de credenciales del cliente.

En el código, hemos definido las dos variables: `$client_id` y `$access_token`, que contienen las credenciales necesarias aportadas por Twitch para autenticar nuestra solicitud a la API de Twitch.

El `client_id` identifica nuestra aplicación, mientras que el `access_token` es un token de acceso que autoriza nuestra solicitud.

Luego, creamos un array `$headers` que contiene los encabezados HTTP necesarios para nuestra solicitud.

Este array incluye el Client-ID y la Authorization con el token de acceso. Estos encabezados aseguran que nuestra solicitud esté autenticada correctamente.

Finalmente, utilizamos `curl_setopt` para configurar la opción de cURL que establece los encabezados HTTP de nuestra solicitud. Esto permite que nuestra solicitud incluya las credenciales necesarias para ser aceptada por la API de Twitch.

```
$client_id = "ynbfszlhhuo6irowc6zuqlzu8g0f2t";
$access_token = "32tleomygkitibfffxh0l0xc1ag2dmw";
$api_url = "https://api.twitch.tv/helix/streams?first=10";

$headers = [
    "Client-ID: $client_id",
    "Authorization: Bearer $access_token"
];

$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, $api_url);
curl_setopt($ch, CURLOPT_HTTPHEADER, $headers);
```

Scripts

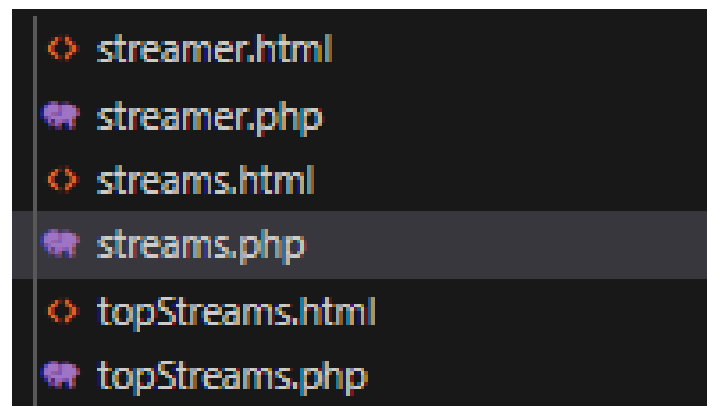
Para la correcta realización de la primera entrega hemos decidido utilizar 3 scripts, uno por cada caso a realizar.

El primer caso consiste en obtener información sobre un streamer mediante la búsqueda por su ID de twitch, para esta funcionalidad hemos creado el script **streamer.php**

El segundo caso consiste en obtener un listado de streams que estuviese en retransmisión en el momento de realizar la petición, en este caso el script creado ha sido **streams.php**

Y para el tercer caso hemos realizado el script **topStreamers.php** el cual obtiene el número de streams con un límite de streams introducido por el usuario en retransmisión en ese momento con información adicional sobre el streamer, siendo esta información su display name y su foto de perfil.

En nuestros scripts se comienza permitiendo peticiones desde cualquier dominio y estableciendo el tipo de contenido como JSON.



streamer.php

Este código en PHP está diseñado para interactuar con la API de Twitch y obtener información sobre un usuario específico utilizando su ID.

Primero, se establecen las credenciales de la aplicación Twitch, como el Client-ID y el Access Token, que son esenciales para autenticar la solicitud a la API. Estos detalles permiten que la API reconozca y autorice la solicitud.

Luego, el código construye la URL de la API de Twitch utilizando el ID de usuario proporcionado. Esta URL se utiliza para realizar la solicitud HTTP y obtener la información del usuario.

La validación de entrada es crucial en este script: verifica si se ha proporcionado un ID de usuario a través de la URL. Si no se proporciona, el script devuelve un mensaje de error en formato JSON indicando que el ID del usuario es necesario: `{"error": "Invalid or missing 'id' parameter."}`.

La solicitud HTTP se realiza utilizando cURL, lo que permite al script obtener el contenido de la respuesta de la API de Twitch.

Si la solicitud es exitosa (código HTTP 200), el código decodifica el contenido JSON de la respuesta y extrae la información relevante del usuario.

El manejo de la respuesta incluye la verificación de los datos obtenidos. Si se encuentran datos, el script devuelve la información del usuario en formato JSON utilizando `json_encode`. En caso de que no se encuentren datos, el usuario no fue encontrado.

streams.php

Este código en PHP está diseñado para interactuar con la API de Twitch y obtener información sobre los streams en vivo.

El código construye la URL de la API de Twitch para obtener los primeros 10 streams. Se configuran los encabezados HTTP necesarios para la solicitud, incluyendo las credenciales de autenticación.

El código utiliza cURL para realizar la solicitud HTTP a la API de Twitch. Si la respuesta es exitosa (código HTTP 200), se procesan los datos y se filtran los campos deseados (nombre del usuario, título del stream y número de espectadores), devolviéndolos en formato JSON.

Si la respuesta es un error (código HTTP 401 o 500), se maneja adecuadamente mostrando mensajes de error específicos en formato JSON. Otros códigos de error también se manejan mostrando un mensaje de error genérico con el código de estado HTTP.

Se configuran los encabezados HTTP necesarios para la solicitud, incluyendo el Client-ID y la Authorization con el token de acceso. A continuación, se inicializa una sesión cURL y se configuran las opciones necesarias, como la URL, los encabezados y la opción para devolver la respuesta como una cadena.

La solicitud cURL se ejecuta y se obtiene la respuesta. Si la respuesta es exitosa (código HTTP 200), se decodifica el contenido JSON de la respuesta y se filtran los campos deseados (nombre del usuario, título del stream y número de espectadores). Estos datos se devuelven en formato JSON.

Si la respuesta es un error (código HTTP 401 o 500), se maneja adecuadamente mostrando mensajes de error específicos en formato JSON. Otros códigos de error también se manejan mostrando un mensaje de error genérico con el código de estado HTTP.

Este proceso asegura que la solicitud a la API de Twitch esté autenticada y que podamos recibir la información solicitada de manera segura y eficiente.

topStreams.php

Este código en PHP está diseñado para interactuar con la API de Twitch y obtener información sobre los streams en vivo, incluyendo detalles adicionales de los usuarios que están transmitiendo.

Luego, el código construye la URL de la API de Twitch para obtener los primeros 10 streams. Se configuran los encabezados HTTP necesarios para la solicitud, incluyendo las credenciales de autenticación.

El código utiliza cURL para realizar la solicitud HTTP a la API de Twitch. Si la respuesta es exitosa (código HTTP 200), se procesan los datos y se filtran los campos deseados (ID del usuario, título del stream y número de espectadores). A continuación, se realiza una segunda solicitud a la API de Twitch para obtener información adicional sobre los usuarios (nombre de usuario y URL de la imagen de perfil).

El script crea un array asociativo con la información del usuario y enriquece los datos de los streams con esta información adicional. Luego, ordena los streams según el número de espectadores, en orden ascendente o descendente, según se especifique en la URL. También permite limitar el número de streams devueltos.

Si la respuesta es un error (código HTTP 400, 401, 404 o 500), se maneja adecuadamente mostrando mensajes de error específicos en formato JSON. Otros códigos de error también se manejan mostrando un mensaje de error genérico con el código de estado HTTP.

Este proceso asegura que la solicitud a la API de Twitch esté autenticada y que podamos recibir la información solicitada de manera segura y eficiente, incluyendo detalles adicionales sobre los usuarios que están transmitiendo.

Código streamer.php

```
<?php
require '../api/crearToken.php';

header("Access-Control-Allow-Origin: *");
header("Content-Type: application/json");

// Verificar si 'id' está en la URL
if (!isset($_GET['id']) || empty($_GET['id'])) {
    echo json_encode(["error" => "Invalid or missing 'id' parameter."]);
    exit;
}

// Obtener el token
$credentials = obtenerToken();

// Manejo de errores si no se obtuvo el token
if (isset($credentials['error'])) {
    echo json_encode(["error" => "Failed to obtain access token", "details" => $credentials]);
    exit;
}

$client_id = $credentials['client_id'];
$access_token = $credentials['access_token'];

$user_id = $_GET['id'];
$api_url = "https://api.twitch.tv/helix/users?id=$user_id";

// Configurar encabezados
$headers = [
    "Client-ID: $client_id",
    "Authorization: Bearer $access_token"
];

$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, $api_url);
curl_setopt($ch, CURLOPT_HTTPHEADER, $headers);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, true);

$response = curl_exec($ch);
$http_code = curl_getinfo($ch, CURLINFO_HTTP_CODE);
curl_close($ch);

// Manejo de respuestas
if ($http_code == 200) {
    $data = json_decode($response, true);
    if (!isset($data["data"][0])) {
        echo json_encode(["error" => "User not found."]);
        exit;
    }

    $streamer = $data["data"][0];
    $resultado = [
        "id" => $streamer["id"],
        "login" => $streamer["login"],
        "display_name" => $streamer["display_name"],
        "broadcaster_type" => $streamer["broadcaster_type"],
        "description" => $streamer["description"],
        "profile_image_url" => $streamer["profile_image_url"],
        "view_count" => $streamer["view_count"],
        "created_at" => $streamer["created_at"]
    ];

    echo json_encode($resultado, JSON_PRETTY_PRINT);
} else {
    echo json_encode(["error" => "Unexpected error", "status" => $http_code, "response" => json_decode($response, true)]);
}
?>
```


Código streams.php

```
<?php

require '../api/crearToken.php';

header("Access-Control-Allow-Origin: *"); // Permitir peticiones desde cualquier dominio
header("Content-Type: application/json");

// Obtener el token
$credentials = obtenerToken();

// Manejo de errores si no se obtuvo el token
if (isset($credentials['error'])) {
    echo json_encode(["error" => "Failed to obtain access token", "details" => $credentials]);
    exit;
}

$client_id = $credentials['client_id'];
$access_token = $credentials['access_token'];

$sapi_url = "https://api.twitch.tv/helix/streams?first=10";

$headers = [
    "Client-ID: $client_id",
    "Authorization: Bearer $access_token"
];

$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, $sapi_url);
curl_setopt($ch, CURLOPT_HTTPHEADER, $headers);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, 1);
$response = curl_exec($ch);
$http_code = curl_getinfo($ch, CURLINFO_HTTP_CODE);
curl_close($ch);

// Manejo de respuestas según código HTTP
if ($http_code == 200) {
    // Si la respuesta es exitosa, procesamos los datos
    $data = json_decode($response, true);

    if (isset($data["data"])) {
        echo json_encode(["error" => "No se encontraron streams"]);
        exit;
    }

    // Filtramos los campos deseados
    $streams_filtrados = array_map(function ($stream) {
        return [
            "nombre" => $stream["user_name"],
            "title" => $stream["title"],
            "espectadores" => $stream["viewer_count"]
        ];
    }, $data["data"]);

    // Mostrar datos en formato JSON
    echo json_encode($streams_filtrados, JSON_PRETTY_PRINT);
} elseif ($http_code == 401) {
    // Si la respuesta es 401, token inválido o expirado
    echo json_encode(["error" => "RESPONSE 401: Unauthorized. Twitch access token is invalid or has expired."]);
} elseif ($http_code == 500) {
    // Si la respuesta es 500, error interno del servidor
    echo json_encode(["error" => "RESPONSE 500: Internal Server Error"]);
} else {
    // Manejo de otros códigos de error
    echo json_encode(["error" => "Unexpected error", "status" => $http_code]);
}

?>
```

Código topStreams.php

```
<?php
require '../api/crearToken.php';

header("Access-Control-Allow-Origin: *"); // Permitir peticiones desde cualquier dominio
header("Content-Type: application/json");

// Obtener el token
$credentials = obtenerToken();

// Manejo de errores si no se obtuvo el token
if (isset($credentials['error'])) {
    echo json_encode(["error" => "Failed to obtain access token", "details" => $credentials]);
    exit;
}

$limit = $_GET['limit'];
$client_id = $credentials['client_id'];
$access_token = $credentials['access_token'];
$api_url = "https://api.twitch.tv/helix/streams?first=$limit";

$headers = [
    "Client-ID: $client_id",
    "Authorization: Bearer $access_token"
];

$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, $api_url);
curl_setopt($ch, CURLOPT_HTTPHEADER, $headers);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, 1);
$response = curl_exec($ch);
$http_code = curl_getinfo($ch, CURLINFO_HTTP_CODE);
curl_close($ch);

// Manejo de respuestas según código HTTP
if ($http_code == 200) {
    // Si la respuesta es exitosa, procesamos los datos
    $data = json_decode($response, true);

    if (!isset($data["data"])) {
        echo json_encode(["error" => "No se encontraron streams"]);
        exit;
    }

    // Filtramos los campos deseados
    $streams_filtrados = array_map(function ($stream) {
        return [
            "user_id" => $stream["user_id"],
            "title" => $stream["title"],
            "espectadores" => $stream["viewer_count"]
        ];
    }, $data["data"]);

    // Obtener información adicional del usuario
    $user_ids = array_column($streams_filtrados, 'user_id');
    $user_ids_str = implode('&id=', $user_ids);
    $user_api_url = "https://api.twitch.tv/helix/users?id=" . $user_ids_str;
```

```

$ch = curl_init();
curl_setopt($ch, CURLOPT_URL, $user_api_url);
curl_setopt($ch, CURLOPT_HTTPHEADER, $headers);
curl_setopt($ch, CURLOPT_RETURNTRANSFER, 1);
$user_response = curl_exec($ch);
curl_close($ch);

$user_data = json_decode($user_response, true);

// Crear un array asociativo con la información del usuario
$user_info = [];
foreach ($user_data['data'] as $user) {
    $user_info[$user['id']] = [
        'display_name' => $user['display_name'],
        'profile_image_url' => $user['profile_image_url']
    ];
}

// Enriquecer los streams con la información del usuario
$streams_enriquecidos = array_map(function ($stream) use ($user_info) {
    return array_merge($stream, $user_info[$stream['user_id']]);
}, $streams_filtrados);

// Ordenar los streams según el número de espectadores
$order = isset($_GET['order']) ? $_GET['order'] : 'asc';
usort($streams_enriquecidos, function ($a, $b) use ($order) {
    if ($order == 'asc') {
        return $a['espectadores'] <=> $b['espectadores'];
    } else {
        return $b['espectadores'] <=> $a['espectadores'];
    }
});

// Mostrar datos en formato JSON
echo json_encode($streams_enriquecidos, JSON_PRETTY_PRINT);
} elseif ($http_code == 400) {
    // Si la respuesta es 400, solicitud incorrecta
    echo json_encode(["error" => "RESPONSE 400: Bad Request"]);
} elseif ($http_code == 401) {
    // Si la respuesta es 401, token inválido o expirado
    echo json_encode(["error" => "RESPONSE 401: Unauthorized. Twitch access token is invalid or has expired."]);
} elseif ($http_code == 500) {
    // Si la respuesta es 500, error interno del servidor
    echo json_encode(["error" => "RESPONSE 500: Internal Server Error"]);
} else {
    // Manejo de otros códigos de error
    echo json_encode(["error" => "Unexpected error", "status" => $http_code]);
}
}
?>

```

Hosting

Para poder desplegar los servicios al principio optamos por obtener un dominio de manera gratuita, pero debido a varios problemas que nos han ido sucediendo y que ya explicaremos más tarde al final optamos por adquirir un dominio mediante el servicio que proporciona la compañía [IONOS](#).

Gracias a IONOS pudimos adquirir un dominio con el nombre de nuestro grupo “Heropass” a un precio bastante asequible, teniendo también las ventajas de un servicio de hosting, ya que lo que nosotros buscábamos era poder subir nuestros propios archivos que pudiesen dar las funcionalidades que nosotros buscábamos al proyecto, otras empresas lo único que ofrecían para desplegar el proyecto era la creación de una web mediante una IA pero no ofrecían la posibilidad de poder subir nuestros propios ficheros.

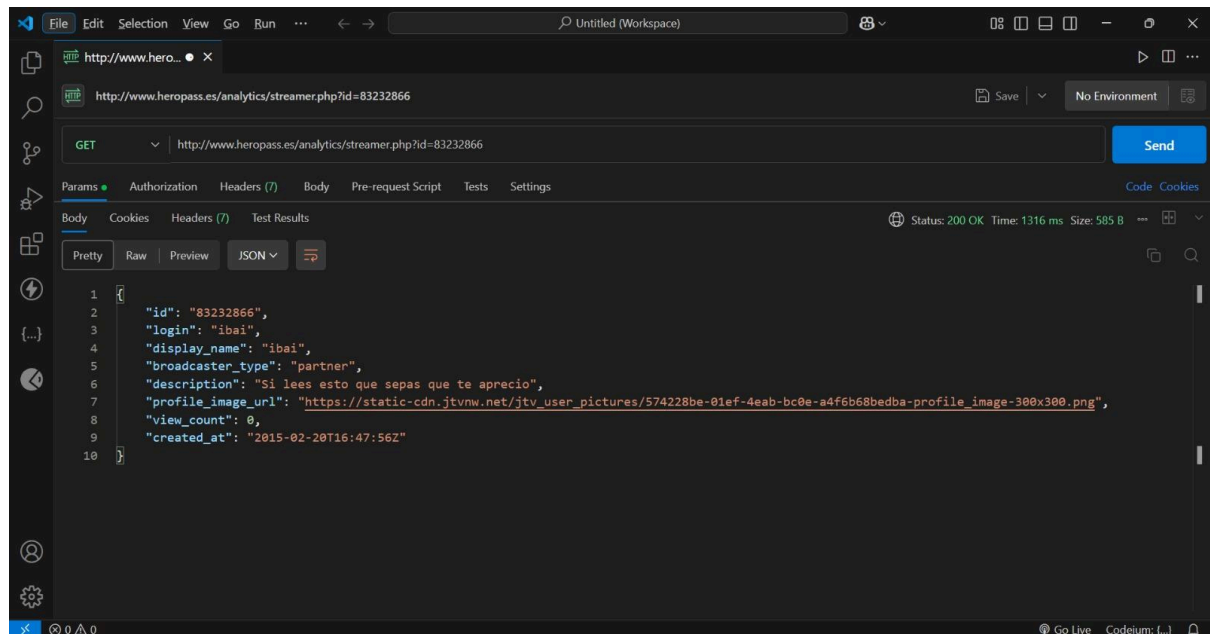
También buscamos que nuestro proyecto fuese de fácil acceso ya que no solo buscamos un lugar fiable donde desplegarlo sino que en un futuro nos pudiese servir como referencia para poder mostrar proyectos realizados durante nuestro curso académico y fuese posible que empresas pudiesen valorar nuestra experiencia.

Por lo que nuestro proyecto está visible aquí: [Heropass](#)

Códigos de respuesta y peticiones

Códigos 200

Caso 1

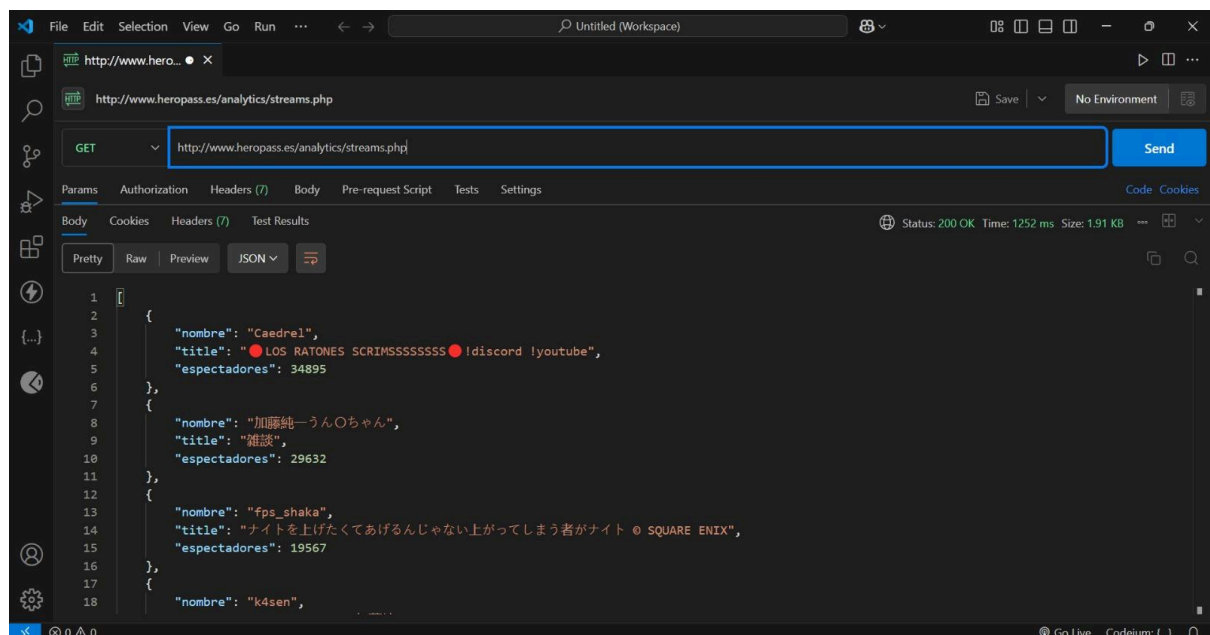


```
GET http://www.heropass.es/analytics/streamer.php?id=83232866
```

Status: 200 OK Time: 1316 ms Size: 585 B

```
{
  "id": "83232866",
  "login": "ibai",
  "display_name": "ibai",
  "broadcaster_type": "partner",
  "description": "Si lees esto que sepas que te aprecio",
  "profile_image_url": "https://static-cdn.jtvnw.net/jtv_user_pictures/574228be-01ef-4eab-bc0e-a4f6b68bedba-profile_image-300x300.png",
  "view_count": 0,
  "created_at": "2015-02-20T16:47:56Z"
}
```

Caso 2

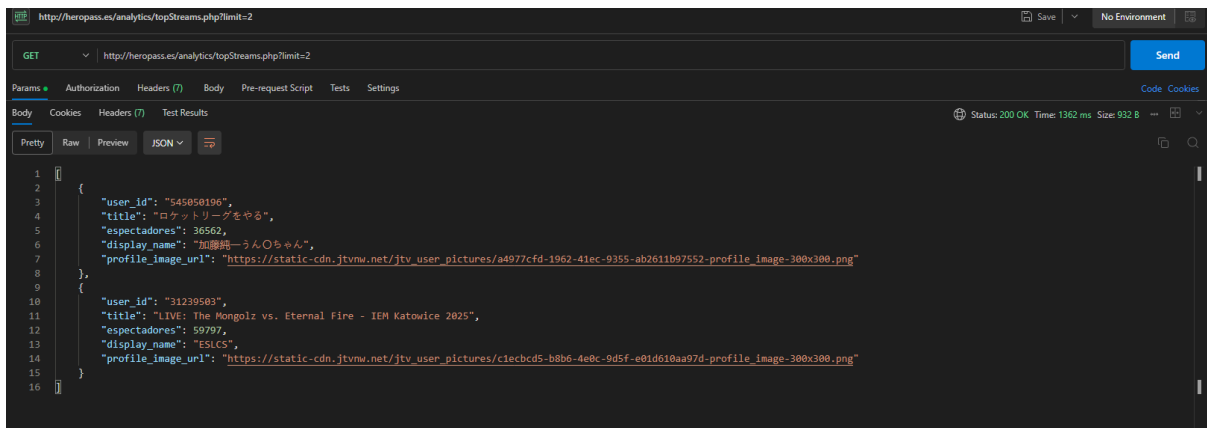


```
GET http://www.heropass.es/analytics/streams.php
```

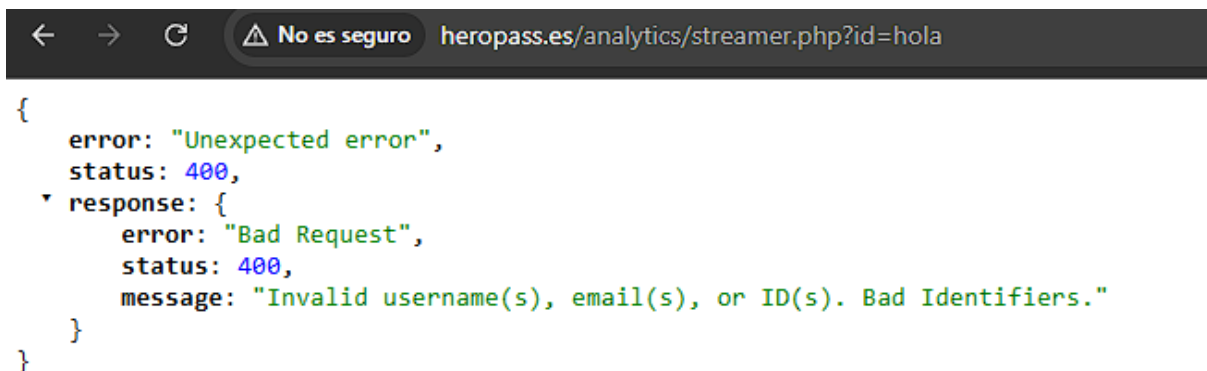
Status: 200 OK Time: 1252 ms Size: 1.91 KB

```
[
  {
    "nombre": "Caedrel",
    "title": "LOS RATONES SCRIMSSSSSSS !discord !youtube",
    "espectadores": 34895
  },
  {
    "nombre": "加藤純一うんちゃん",
    "title": "雑談",
    "espectadores": 29632
  },
  {
    "nombre": "fps_shaka",
    "title": "ナイトを上げたくてあげるんじゃない上がってしまう者がナイト @ SQUARE ENIX",
    "espectadores": 19567
  },
  {
    "nombre": "k4sen",
    "title": "",
    "espectadores": 0
  }
]
```

Caso 3

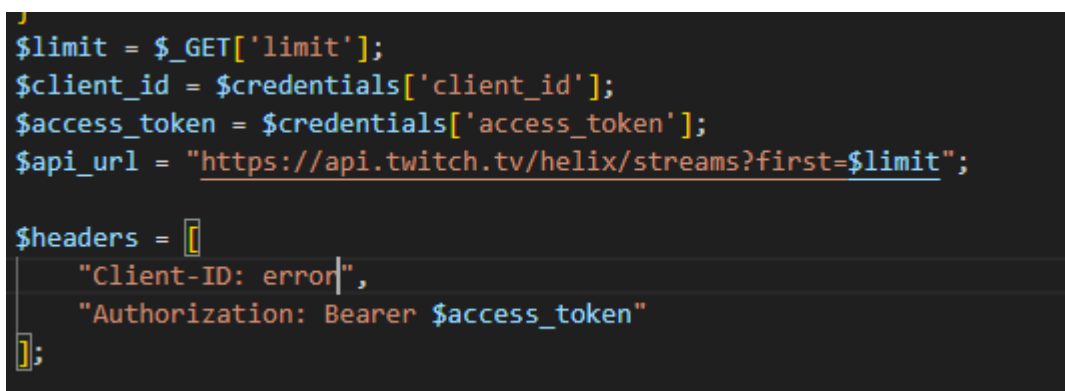


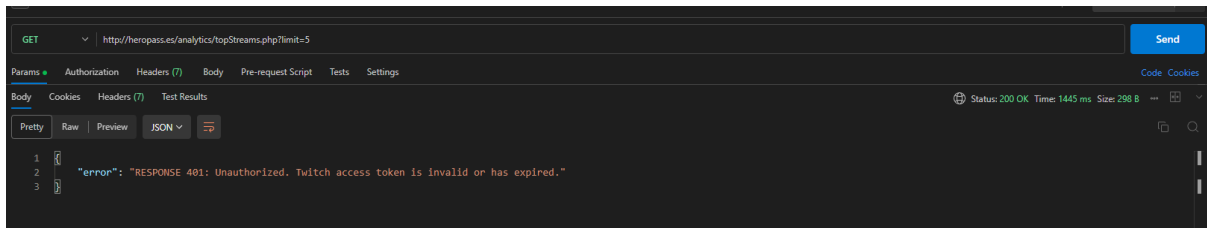
Código 400



En este caso si observamos el valor del parámetro de id que le hemos pasado es un string “hola” lo cual es incorrecto ya que nosotros buscamos un id numérico por lo que nos va a devolver el código 400 referido a petición errónea.

Código 401

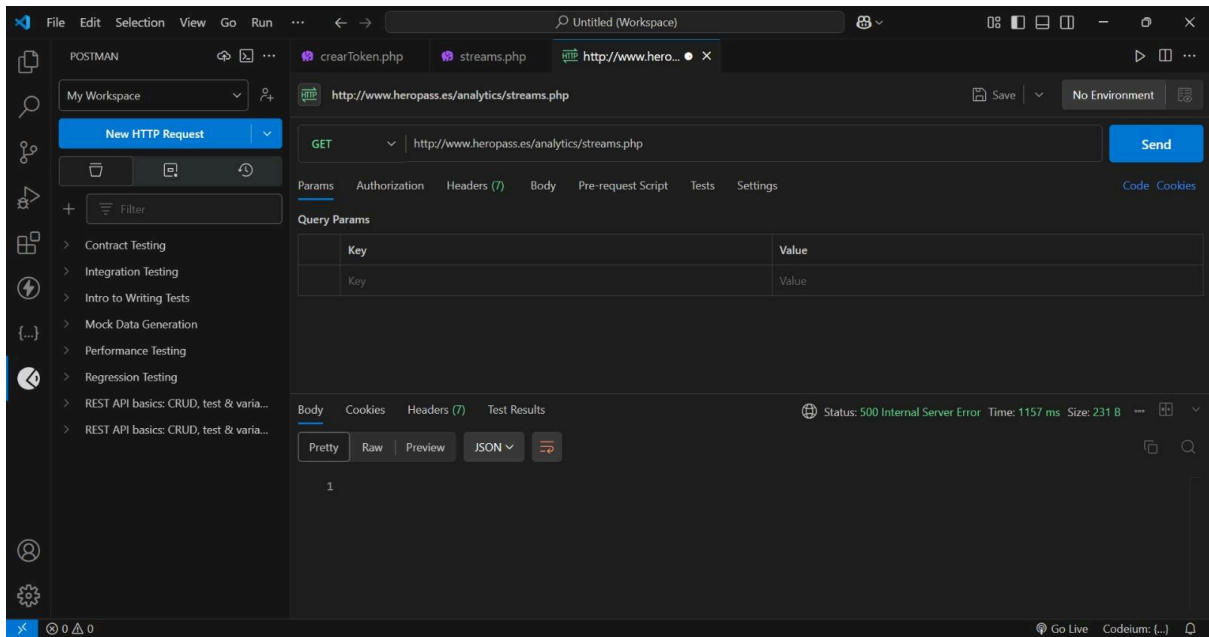




En nuestro caso no tendríamos error 401 en nuestro proyecto ya que tenemos nuestras variables ya establecidas, pero en el caso de que al usuario se le solicitase su identificador de cliente y este lo introdujese erróneamente, saldría este código 401.

Código 500

```
public_html > analytics > streams.php
1  <?php
2
3  // Manejo de errores si no se obtuvo el token
4  if (isset($credentials['error'])) {
5      echo json_encode(["error" => "Failed to obtain access token", "details" => $credentials]);
6      exit;
7  }
8
9  $client_id = $credentials['client_id'];
10 $access_token = $credentials['access_token'];
11
12 $api_url = "https://api.twitch.tv/helix/streams?first=10";
13
14 echo |
15
16 $headers = [
17     "Client-ID: $client_id",
18     "Authorization: Bearer $access_token"
19 ];
```



En este caso como se puede observar hay un fallo en el código y es que hay un echo sin cerrar, lo que hace que no funcione correctamente el php y proporcione un código 500 de error.

Problemas encontrados y soluciones

Dominio web:

De primeras intentamos utilizar varias páginas web gratuitas para alojar nuestra API, con GithubPages tuvimos problemas ya que al subir los archivos nos dimos cuenta de que no podíamos utilizar archivos .php. Después probamos con InfinityFree, una página que permitía obtener dominios de forma gratuita, pero tuvimos problemas a la hora de realizar las peticiones ya que muchas veces fallaban.

Finalmente optamos por comprar un dominio mediante Ionos lo cual nos quitó todos los problemas anteriores y nos permitió hacer todas las pruebas para poner en producción nuestra API correctamente.

Comprensión incorrecta de la práctica:

No conseguimos comprender del todo como debía funcionar la API, y empezamos a hacer una página web con html con los que había que interactuar para obtener las peticiones y los mostramos formateados en html.

Cuando comprendimos que hacer con la API, no tuvimos que realizar grandes cambios ya que ya teníamos los ficheros php que realizaban las peticiones.

Refresh token:

Tuvimos problemas a la hora de refrescar el token, de primeras utilizamos un único token. Pero más adelante nos dimos cuenta que ese token caducaría, y no lo podríamos utilizar siempre en el código.

Teníamos dos opciones, la primero era usar un refresh token, comprobar si el token que utilizábamos actualmente estaba caducado y si era así generar uno nuevo y guardarlo, y la segunda que finalmente fue la que utilizamos, que es generar un nuevo acces token cada vez que se realiza una petición en nuestra API, esta nos pareció más sencilla de implementar.